

Lecture 7. DevOps

Concept and Practice



Lee Youngkon



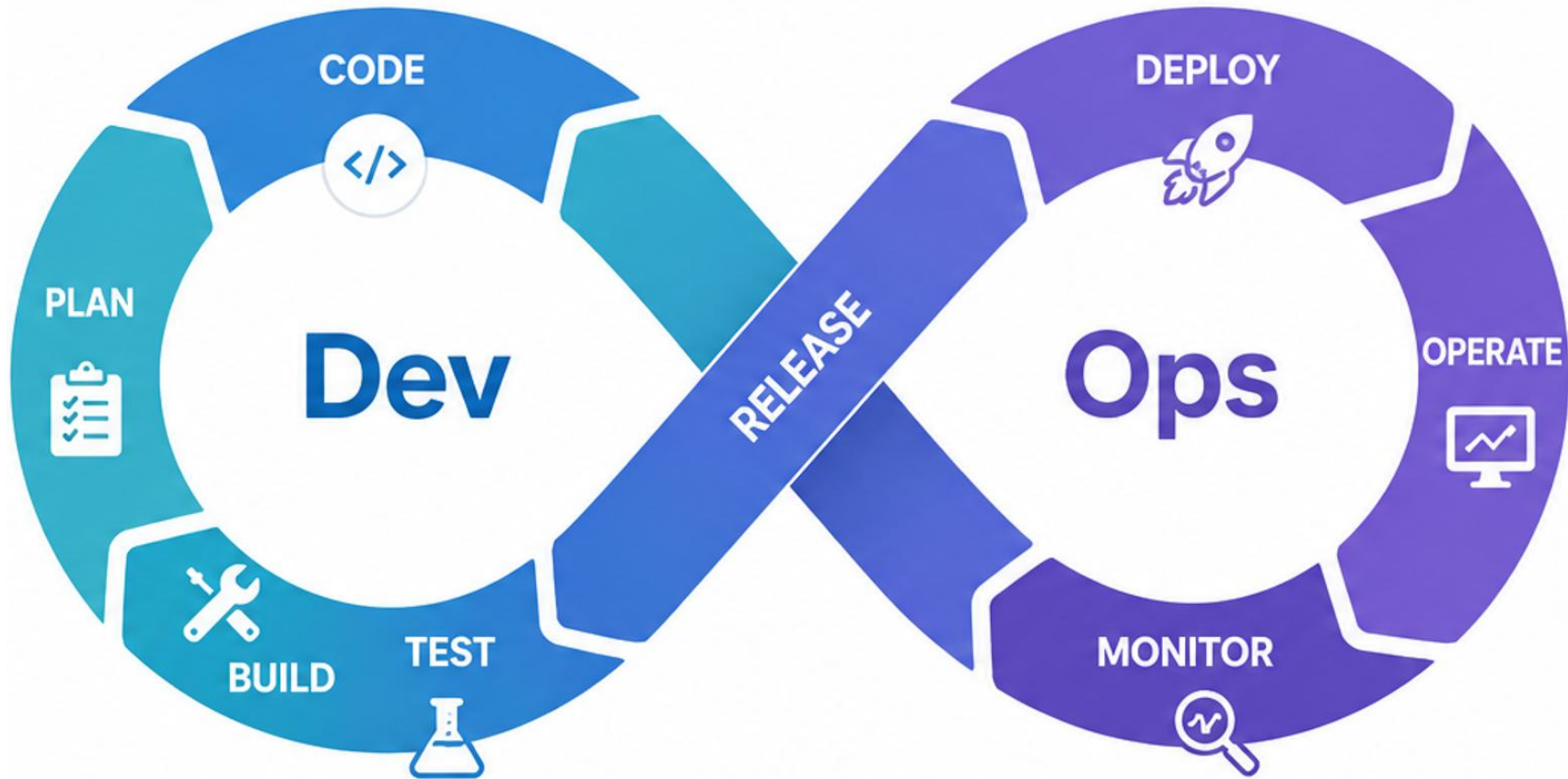
DevOps Background

- ✓ Existing Development Method
 - Development team → code writing
 - Operations Team → Service Operations
- ✓ problem
 - Slow deployment
 - Separation of development and operational responsibilities
 - Failure Response Delay
 - Environmental inconsistency



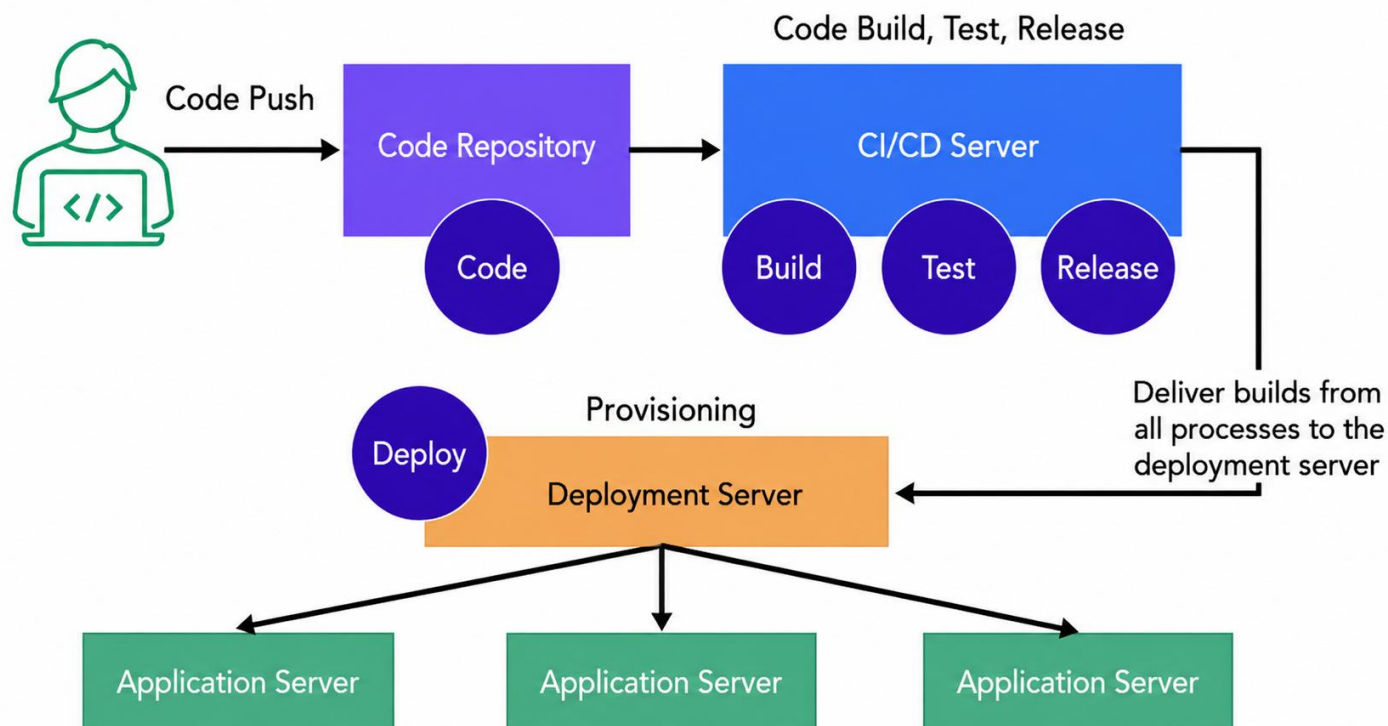
- ✓ DevOps Goals
 - Development + Operational Integration
 - Automation-based deployment
 - Fast service delivery
 - stable operation
 - Maintaining Security

DevOps Concept



DevOps Components

- ✓ Continuous Integration (CI)
- ✓ Continuous Delivery (CD)
- ✓ Continuous Monitoring
- ✓ Automation
- ✓ Collaboration



1. the prohibition of reckless expansion of automation

- ✓ CI/CD, IaC, deployment automation required
- ✓ Unverified automation can spread failures on a large scale
- ✓ The following must be provided
 - Test Automation
 - Approval Process
 - Rollback strategy
 - Blue/Green Deployment
 - Deploy Canary

2. Internalize Security (DevSecOps)

- ✓ Don't hardcode secret keys, DB passwords, certificates
- ✓ Compliance with
 - Docker Secret / Kubernetes Secret
 - Vault
 - Scan for Image Vulnerabilities
 - SAST/DAST
 - RBAC Minimum Privilege Principle

- ❖ Static Application Security Testing (SAST): A method of statically analyzing source code, binary, or build results without running them to find security vulnerabilities. ex. SonarQube
- ❖ Dynamic Application Security Testing (DAST): How to test running applications (Web/API) from the perspective of external attackers. ex. Buff Suit

3. Do Not Deploy Without Monitoring

- ✓ "Successful distribution" $\neq$ "Normal service"
- ✓ Requirements:
 - Collect Logs (ELK, Loki)
 - Metric (Prometheus)
 - Tracing (Jaeger)
 - Alert manager

4. Maintain environmental consistency

- ✓ Minimize development/testing/operational environmental differences
- ✓ Docker/Kubernetes/IaC(Terraform) active utilization

5. Prevent data loss

- ✓ Most dangerous when changing the DB
- ✓ Compliance with
 - Managing Backup Schema Versions
 - Validation of migration
 - PV/PVC Strategy
 - Rehearsal

6. Management of inter-service dependencies

- ✓ API version management
- ✓ Circuit Breaker
- ✓ Retry Policy
- ✓ Timeout
- ✓ Message duplication (Idempotency)

- ❖ MTTR (Mean Time To Recovery)
- ❖ Change Failure Rate: Percentage of deployment changes causing failures/rollbacks/hotfixes

7. Reliability over deployment speed

- ✓ Frequent deployment itself is not the goal
- ✓ KPI
 - MTTR
 - Deployment Frequency
 - Change Failure Rate

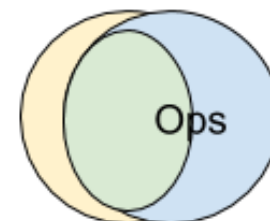
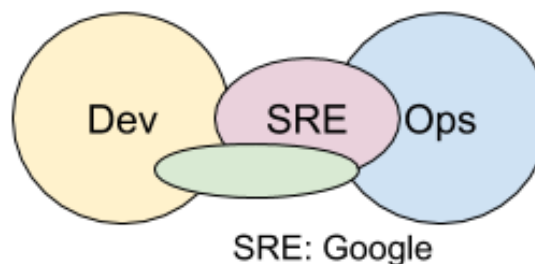
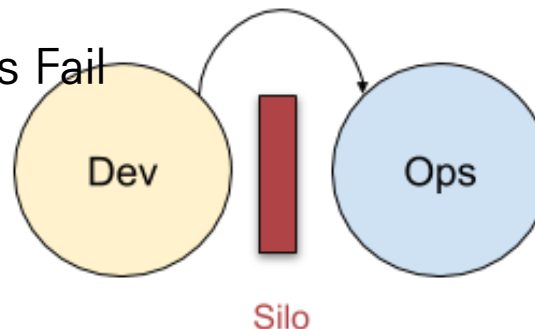
} DORA Metrics(DevOps Research and Assessment)

8. organizational culture issues

✓ Dev vs Ops Separation Culture Continues Fail

✓ Key points:

- joint responsibility
- collaboration
- Standardization
- documentation



Fully Shared: Netflix, "Full Stack"

❖ SRE(Site Reliability Engineering)

- Engineering framework to deploy DevOps centered on operational stability
- Responsibilities for managing system operations, availability, scalability, and recoverability
- Resolve operations (Ops) with code and automation, not by hand

9. Cost Management

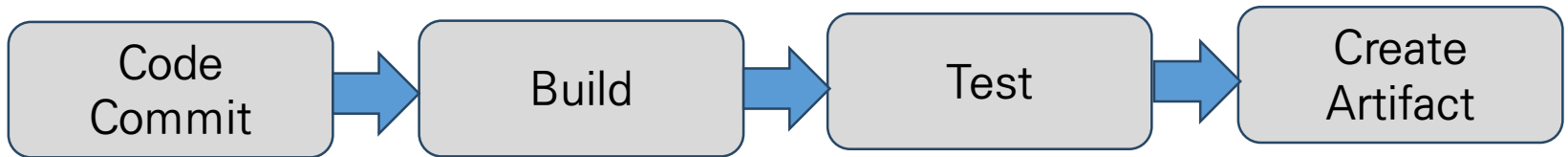
- ✓ Cloud/K8s is expensive when it is used carelessly
- ✓ precautions
 - Excessive resource request
 - Unused Pod
 - Over-save logs
 - GPU/storage waste

10. Disaster Recovery Plan

- ✓ Disaster must happen
- ✓ Recovery plan:
 - DR Scenario, Multiple AZs, Backup, Restore Test, Operational Manual

Continuous Integration (CI)

- ✓ Automatically build and test developer-generated code

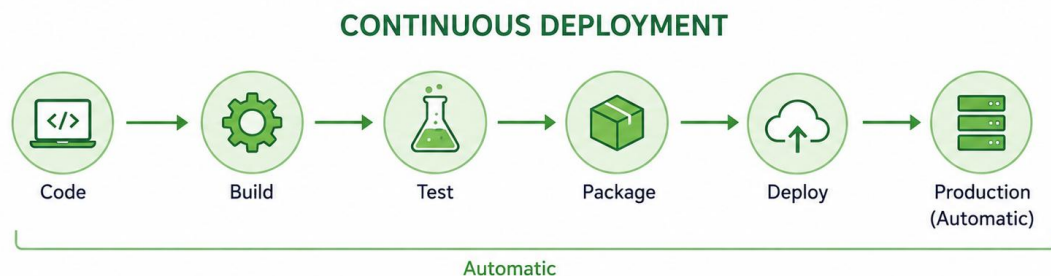
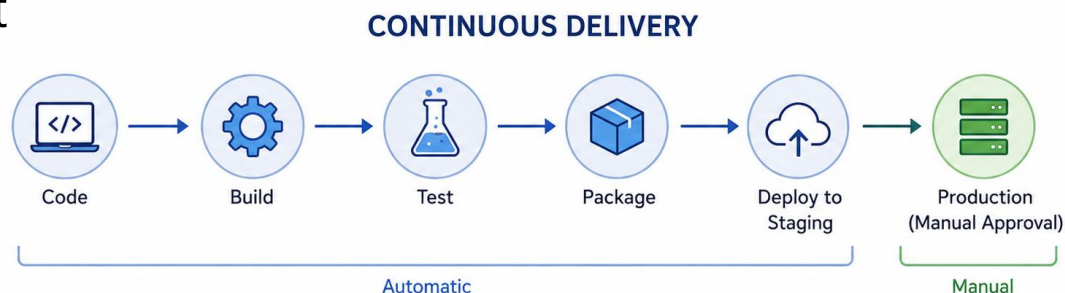


- ✓ Representative Tools: Jenkins, GitHub Action, GitLab CI, Circle CI

Continuous Delivery / Deployment

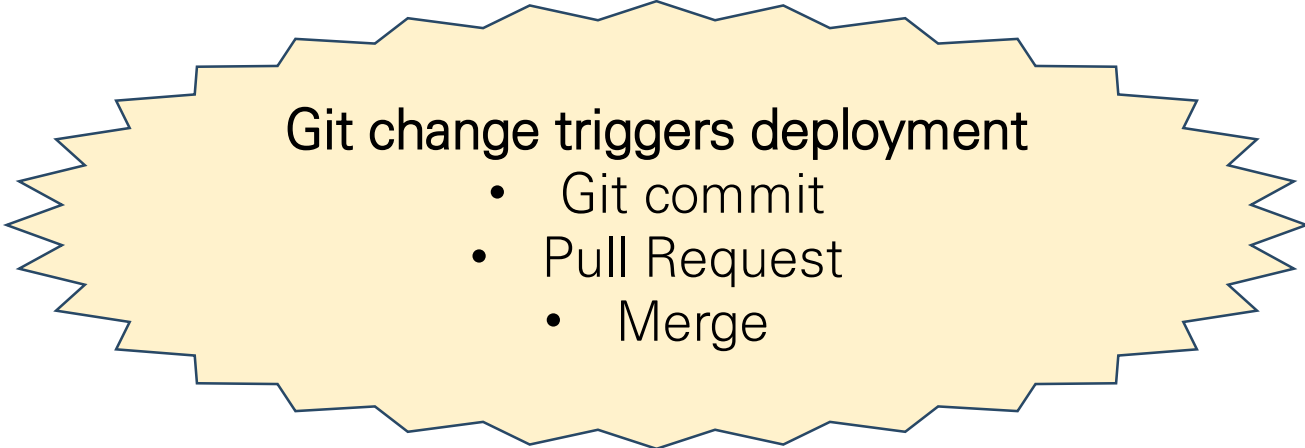
- ✓ Continuous Delivery
 - Auto Build + Test
 - Manually Approve Deployment

- ✓ Continuous Deployment
 - Automatic build
 - Automatic test
 - Automated Deployment



Define GitOps

- ✓ Operational approach to managing infrastructure and applications using Git as a single source of truth
 - Git = System State = Baseline document for current operational state



Git change triggers deployment

- Git commit
- Pull Request
- Merge

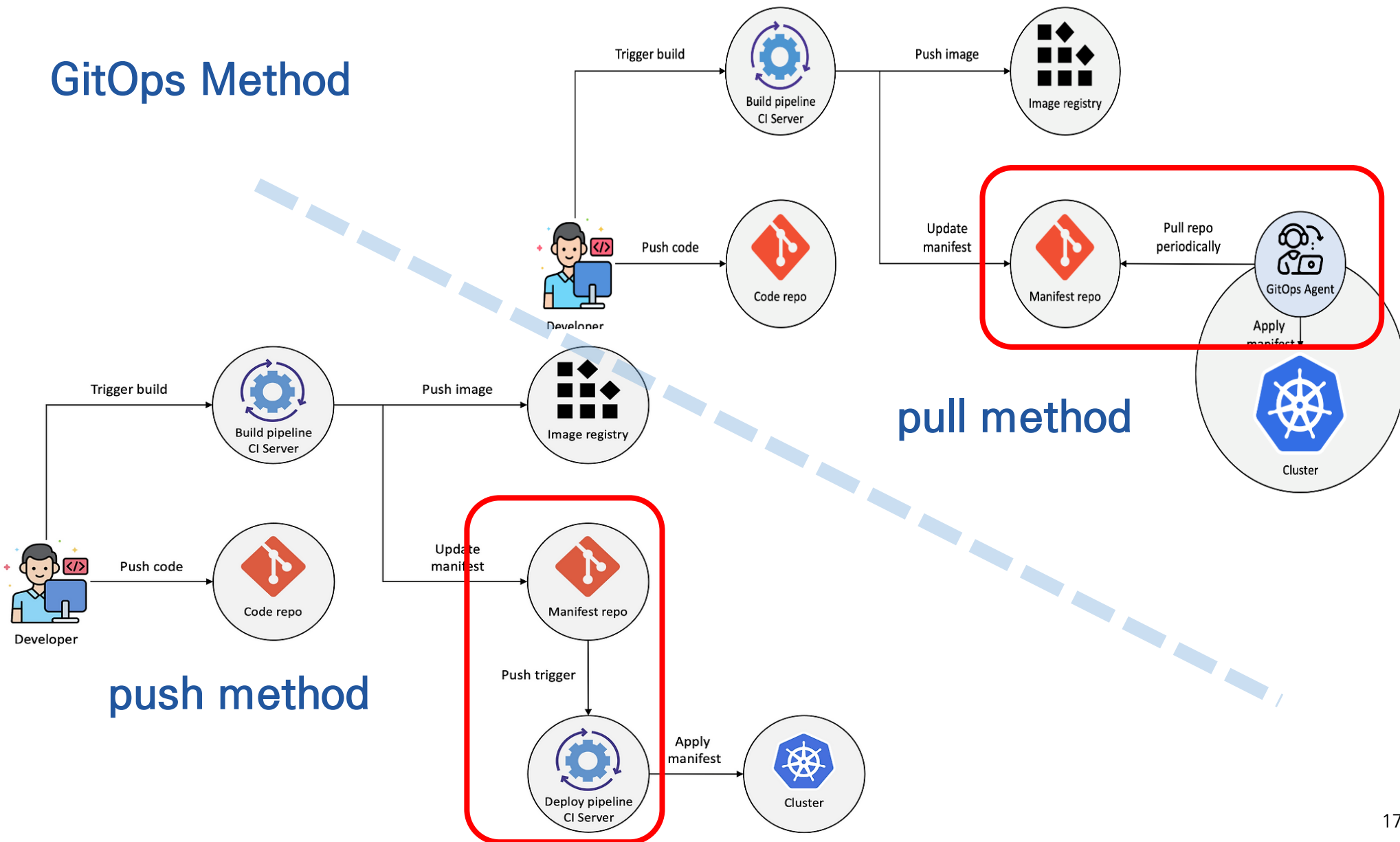
GitOps Core Principles

- ✓ declarative composition
 - Infrastructure as Code
 - Kubernetes YAML
 - Helm
- ✓ Git Single Source
 - Save all states Git
 - deployment.yaml
 - service.yaml
- ✓ automatic synchronization
 - Git Change → System Reflections
- ✓ Pull method
 - System monitors Git
 - Cluster → Git

GitOps Core Tools

Way	Optional/ Require	Role	note
Kubernetes YAML	Optional	Define the most basic deployments	
Helm Chart	Optional	Template-based deployment	Complex service packaging, variable management, version management
Kustomize	Optional	Overlay by dev/stage/prod environment	Maintain YAML source. Good readability. Simple environmental separation clear
Terraform	Optional	Create Cloud/Infrastructure Resources	Utilize when running IaC
Argo CD / Flux	Required	Synchronize Git and Cluster States	Flux takes advantage of declarative automation and image update automation more strongly than UI

GitOps Method



GitOps Components

- ✓ Git Repository
- ✓ CI Pipeline
- ✓ Container Registry
- ✓ GitOps Controller
- ✓ Kubernetes Cluster

Git Repository

- ✓ Structure for storing project files and all changes
- ✓ Simple File Folder + Change History DB

Function	Description
Version Control	Save Change History
Branching	Quarter by function
Merge	code integration
Rollback	Recovery of previous state
Collaboration	team collaboration
Audit	Track who changed what

Git Repository

category	GitHub	GitLab
SaaS	Industry-leading SaaS platform	Mature SaaS Platform
On-premise	Enterprise requires separate deployment	Full Self-host Support
CI/CD	Extensions based on GitHub Actions	Basic Integrated Platform
DevSecOps Integration	Center for external tool linkage	Basic integration of security features
UI/UX	User-friendly	Feature-driven
open-source ecosystem	the largest ecosystem	a powerful ecosystem of its own
Corporate Security Control	outstanding	Enterprise-class control
Cost	SaaS-centric, Enterprise costs high	Self-host based. cost-effectiveness

CI Pipeline

- ✓ Automated verification process that automatically builds, tests, quality checks, security checks, and more whenever developers push code changes to Git
- ✓ Fast and secure integration of code changes
 - Code crash early detection
 - Maintaining Quality
 - Ensuring deployment readiness
 - Automated Reliability Enhancements

Git Push
→ GitHub Actions
→ Maven Build
→ JUnit
→ SonarQube
→ Docker Build
→ Harbor Push

Container Registry

- ✓ Central repository for storing, versioning, and deploying Docker/OCI container images
- ✓ After the container build, the container image exists only on the local PC
- ✓ You must be available in the following environments to deploy your production environment
 - Kubernetes
 - Another server
 - CI/CD Development Team

Requires a repository
accessible to all

👉 Container Registry use

Container Registry Features

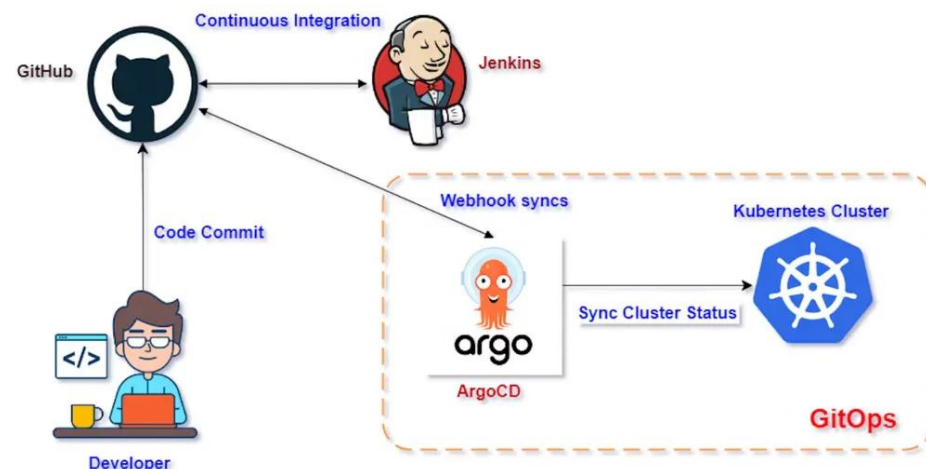
Function	Description
Image Storage	Save Image
Version Tagging	v1.0.1 etc
Access Control	User/Team Rights
Vulnerability Scan	Security vulnerability check
Replication	Multiple environment replication
Signature	Image Signing
Audit	Track who distributed it

Container Registry Comparison

category	Docker Hub	Harbor
Operation type	SaaS-based	Self-host / Private Registry
a public image	Global Largest	restrictive
Private Registry	Support	Enterprise-grade
On-premise	Enterprise-centric	Full support
security control	Standard level	Advanced Policy Control
Scan for vulnerabilities	Partial Support	Basic Interior
Image Signing	partial support	Policy-based support
RBAC	Managing Default Privileges	fine-grained access control
Replication	restrictive	Multi-site support
Speed	Internet environmental dependence	internal network optimization
Cost control	Subscription-based	Optimizing your own operations
Enterprise DevOps Conformity	a mid-range level	Best for big business/on-premises

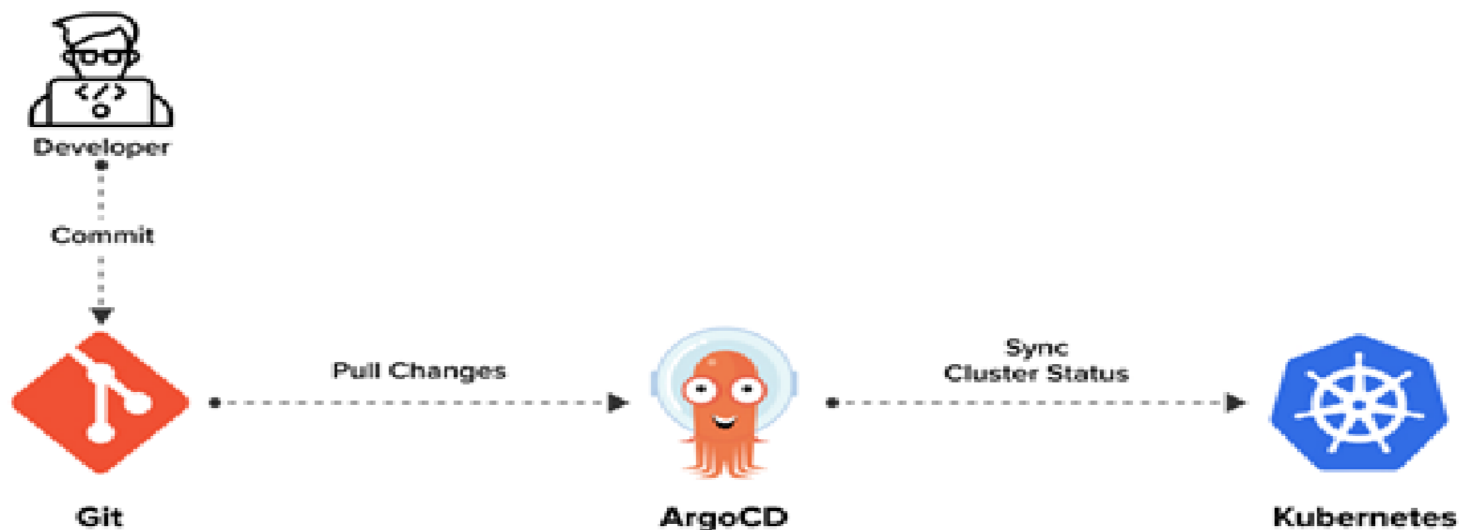
GitOps Controller

- Operational automation engine that continuously compares and automatically matches actual Kubernetes/infrastructure state with desired system state defined in Git repository
- Actual Cluster State $\neq$ Git State $\rightarrow$ Automatic Correction
- GitOps Controller = "Automatic Deployment + Continuous Health Monitoring + Drift Recovery System"
- Representative Tools: Argo CD, Flux



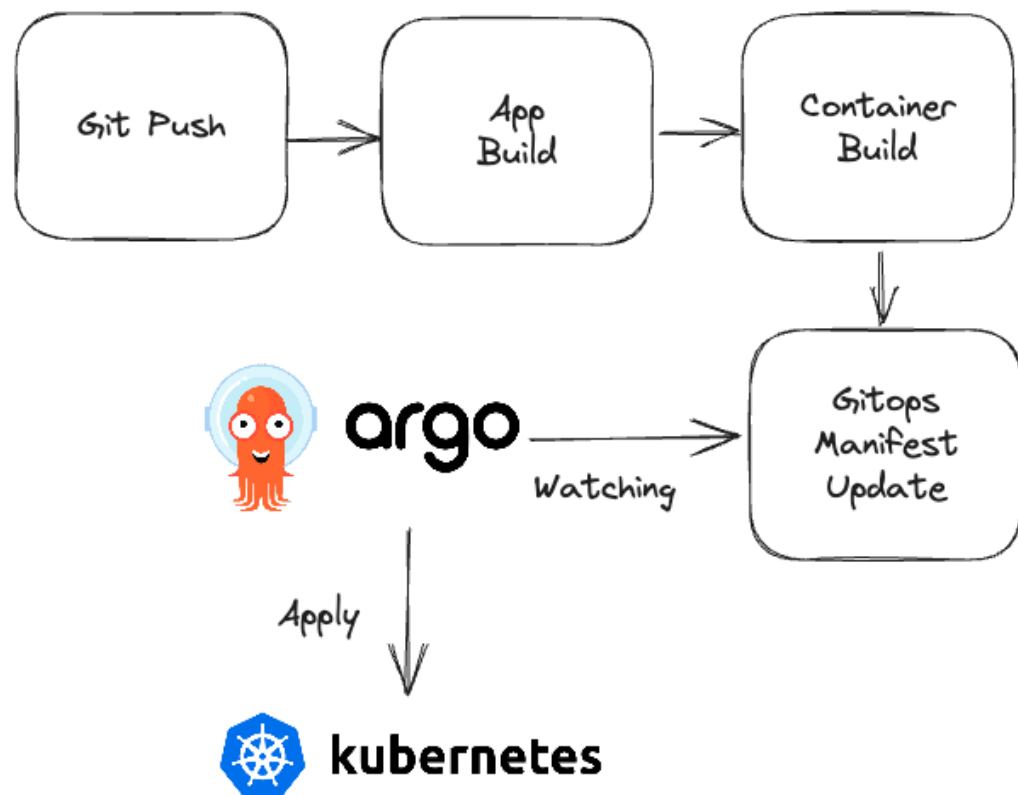
ArgoCD: Git Ops Deployment Tool for Kubernetes

- Automate Kubernetes Deployment
- Git health monitoring
- automatic synchronization
- Rollback Support

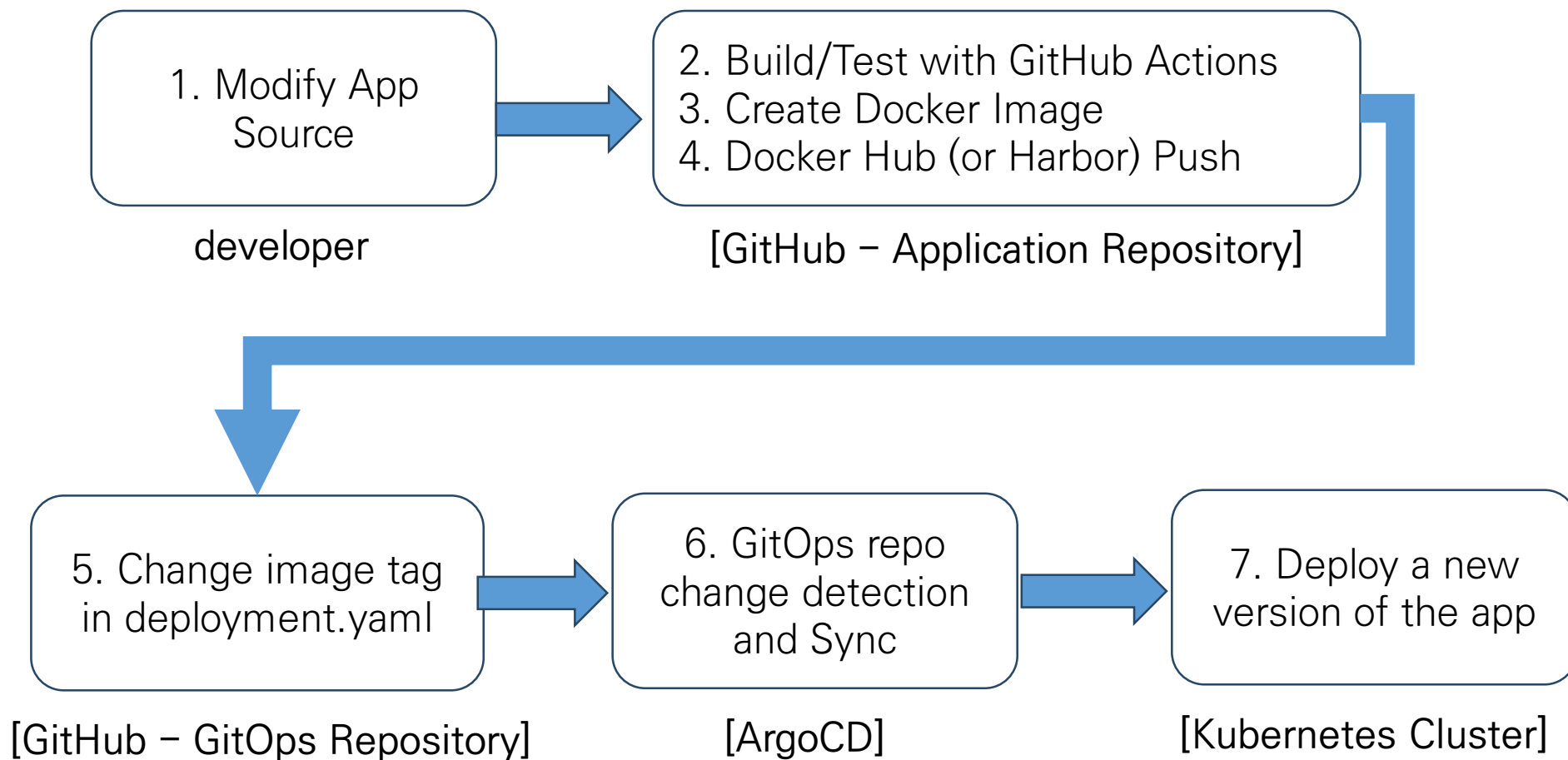


ArgoCD Operation Course

- ✓ Git Repository check
 - deployment.yaml
 - service.yaml
- ✓ Cluster status comparison
 - Desired State
 - Actual State
- ✓ Occurrence of difference
 - Synchronize
- ✓ Deploying Kubernetes



Practice Process



③ Change the manifest

② container image push

① commit & push

④ Manifest change detection

⑤ Apply changes to the cluster

GitHub

Docker Hub

HARBOR

argo

Kubernetes cluster

Cloud provider API

Control Plane

Nodes

Cloud provider API

Control Plane

Nodes

API Server

Cloud Controller Manager (CCM)

Controller Manager

etcd (persistent store)

Initiation

kube-proxy

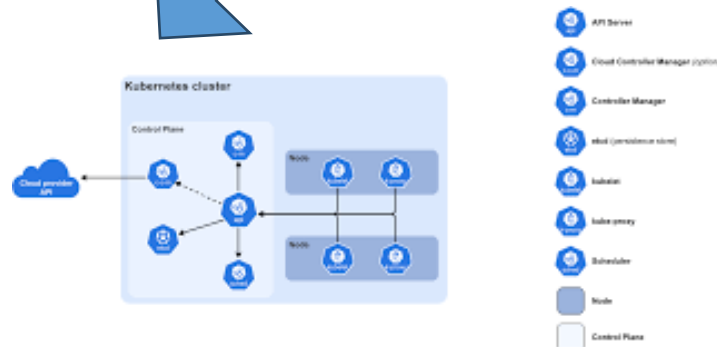
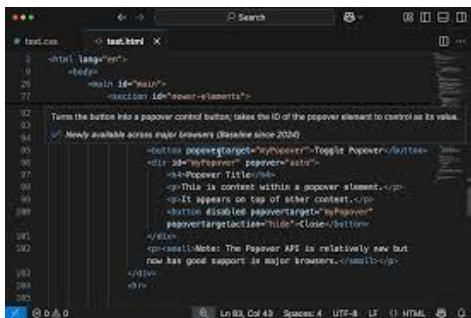
Scheduler

Node

Control Plane

```

1 kind: Deployment
2 metadata:
3   name: test-deploy
4   namespace: default
5 spec:
6   replicas: 1
7   selector:
8     matchLabels:
9       app: test-deploy
10  template:
11    metadata:
12      labels:
13        app: test-deploy
14    spec:
15      containers:
16      - name: test-container
17        image: docker.io/nginx:1.21.0
18        ports:
19        - containerPort: 80
20  
```



kubernetes cluster

Application Repository

Configuration

```
book-api
├── src
│   ├── main
│   │   ├── java/com/example/bookapi
│   │   │   ├── BookApiApplication.java
│   │   │   ├── controller/BookController.java
│   │   │   ├── model/Book.java
│   │   │   └── service/BookService.java
│   │   └── resources
│   │       └── application.yml
├── pom.xml
├── Dockerfile
├── .github
│   └── workflows
│       └── ci.yaml
```

Role

- ✓ Java Code Management
- ✓ Maven Build Test
- ✓ Create Docker Image
- ✓ Registry Push

GitOps Repository

Configuration

```
book-api-gitops
├── base
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
├── overlays
│   └── dev
│       ├── namespace.yaml
│       └── kustomization.yaml
```

Role

- ✓ Managing Kubernetes manifest
- ✓ Managing the deployment version (image tag)
- ✓ Managing settings by environment (dev/prod)
- ✓ Reference repository that ArgoCD refers to

CI/CD for GitOps Repository and App Repository Same or Separation

same

- ① Git Push
- ② Build
- ③ Docker Hub Push
- ④ Modify k8s/deployment.yaml
image tag
- ⑤ Same repo commit
- ⑥ ArgoCD Sync

Separation

- ① App Repo Push
- ② Build
- ③ Docker Hub Push
- ④ Modify GitOps Repo
deployment.yaml
- ⑤ GitOps Repo commit
- ⑥ ArgoCD Sync

When separating the GitOps repository from the App repository

- ✓ GITOPS_TOKEN must be registered in App repo.
- ✓ That way, the app repo can have permission to modify the contents of the GitOps repo.
- ✓ GITHUB_TOKEN is issued according to the procedure below, and it is registered as GITOPS_TOKEN in App repo's secret.
 - ① GitHub login
 - ② Settings
 - ③ Developer settings
 - ④ Personal access tokens
 - ⑤ Fine-grained tokens
 - ⑥ Generate new token

When separating the GitOps repository from the App repository

Only select repositories

Select at least one repository. Max 50 repositories. Also includes public repositories (read-only).

Select repositories 2

Selected 2 repositories.

leeyoungkon/resttemplateserver-sep-gitops	×
leeyoungkon/resttemplateserver-sep	×

Permissions

Choose the minimal permissions necessary for your needs. [Learn more about permissions.](#)

Repositories 4

Account 0

+ Add permissions

Actions
Workflows, workflow runs and artifacts. [Learn more.](#)

Access: Read and write

×

Contents
Repository contents, commits, branches, downloads, releases, and merges. [Learn more.](#)

Access: Read and write

×

When separating the GitOps repository from the App repository

HEALTH

 Healthy

LINKS

IMAGES

`210.90.24.172:8080/camit/yklee2002/resttemplateserver-sep:latest` `mysql:8.0`

SOURCE

EDIT

REPO URL

<https://github.com/leeyoungkon/resttemplateserver-sep-gitops.git>

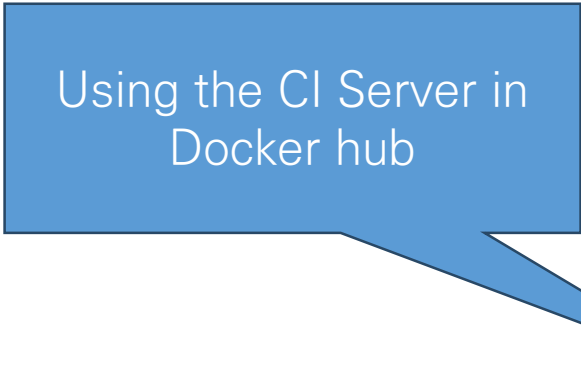
TARGET REVISION

[HEAD](#) 

PATH

[overlays/dev](#) 

CI.yml



Using the CI Server in
Docker hub

```
name: CI/CD Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
env:
```

```
  DOCKERHUB_USERNAME:
```

```
    ${{ secrets.DOCKERHUB_USERNAME }}
```

```
  IMAGE_NAME: resttemplateserver-hub
```

```
jobs:
```

```
  build-and-push:
```

```
    name: Build & Push to Docker Hub
```

```
    runs-on: ubuntu-latest
```

```
  outputs:
```

```
    image_tag: ${{ steps.meta.outputs.tag }}
```

```
steps:
```

```
  - name: Checkout source
```

```
    uses: actions/checkout@v4
```

CI.yml

- name: Set up JDK 17
uses: actions/setup-java@v4
with:
 java-version: '17'
 distribution: 'temurin'
 cache: maven
- name: Grant execute permission for Maven Wrapper
run: chmod +x ./mvnw
- name: Build with Maven
run: ./mvnw clean package -DskipTests
- name: Generate image tag
id: meta
run: echo "tag=\${{ github.sha }}" >> \$GITHUB_OUTPUT
- name: Log in to Docker Hub
uses: docker/login-action@v3
with:
 username: \${{ secrets.DOCKERHUB_USERNAME }}
 password: \${{ secrets.DOCKERHUB_TOKEN }}

CI.yml

```
- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: |
      ${ env.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:
      ${ env.DOCKERHUB_USERNAME }/${ env.IMAGE_NAME }:latest

update-manifest:
  name: Update K8s Manifest for ArgoCD
  runs-on: ubuntu-latest
  needs: build-and-push
```

CI.yml

steps:

- name: Checkout source
uses: actions/checkout@v4
with:
token: \${ secrets.GITHUB_TOKEN }
- name: Update image tag in k8s.yaml
run: |
NEW_IMAGE="\${ env.DOCKERHUB_USERNAME }/\${ env.IMAGE_NAME }:---"
sed -i "s|image: .*\${ env.IMAGE_NAME }.*|image: \${NEW_IMAGE}|g" k8s.yaml
- name: Commit and push manifest update
run: |
git config user.name "github-actions[bot]"
git config user.email "github-actions[bot]@users.noreply.github.com"
git add k8s.yaml
git diff --staged --quiet || git commit -m "ci: update image to ---}"
git push

Replace image tags for
k8s.yaml

Commit modifying the
manifest file

Reasons for Image Tag Replacement by CI.yml (SHA)

- ✓ If you only use the last, latest = mutable
 - Content can be changed with the same tag → Difficult to trace
 - Low Reproducibility
 - Difficulty in recovery

Way	uniqueness	Rollback	traceability
latest	low	difficulty	low
SHA	high	Easy	high

Role classification

CI Role

- ✓ In Application reply:source checkout
- ✓ Maven build/test
- ✓ Docker image build
- ✓ Docker registry push
- ✓ GitOps repo checkout
- ✓ Reflect new image tag
- ✓ GitOps repo commit/push

Role of ArgoCD

- ✓ Monitor GitOps repo changes
- ✓ Compare to desired state and actual state
- ✓ sync when detecting difference
- ✓ Reflect Kubernetes Deployment

ArgoCD.yml

by tool
automatic
generation

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: book-api-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-id/book-api-gitops.git
    targetRevision: HEAD
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: book-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Preparation at Docker hub ⇒ generate token

The screenshot shows the Docker Hub account settings page for user 'yklee2002'. The left sidebar contains navigation links: Home, Hub, Build Cloud, Hardened Images, Scout, Testcontainers Cloud, Docker Desktop, Settings, Account information, Email, Password, 2FA, Personal access tokens (highlighted with a red circle), Connected accounts, Convert, and Privacy. The main content area is titled 'Personal access tokens' and includes a description: 'You can use a personal access token instead of a password for Docker CLI authentication. Create multiple tokens, control their scope, and delete tokens at any time. [Learn more](#)'. Below this is a table of tokens.

Description	Scope	Status	Source	Created	Last used	
my token	Read, Write, Delete	Active	Manual	Mar 12, 2026 at 12:16:16	Mar 12, 2026 at 12:19:02	
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Feb 23, 2026 at 12:56:47	Mar 12, 2026 at 11:38:35	
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Jun 09, 2025 at 10:55:02	Mar 09, 2026 at 12:39:53	Never
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Feb 21, 2026 at 18:21:56	Feb 23, 2026 at 12:30:41	Never
Generated through desktop a...	Read, Write, Delete	Active	Auto-generated	Jan 01, 2026 at 15:55:25	Feb 21, 2026 at 14:18:08	Never
my token3	Read, Write, Delete	Active	Manual	Dec 03, 2025 at 12:02:44	Feb 20, 2026 at 23:32:40	Expired

A dropdown menu is open on the right side of the page, showing options: What's new, My profile, Account settings (circled in red), Billing, and Sign out.

Preparation at Docker hub ⇒ generate token

The screenshot displays the Docker Hub interface for creating a personal access token. The left sidebar shows the user profile 'ykle2002' and a list of navigation options. The 'Personal access tokens' option is highlighted with a red circle. The main content area shows the 'Create access token' form with the following fields:

- Access token description:** A text input field.
- Expiration date:** A dropdown menu set to 'None'.
- Optional:** A section header.
- Access permissions:** A dropdown menu set to 'Read, Write, Delete', which is circled in red.

Below the form, there is a note: 'Read, Write, Delete tokens allow you to manage your repositories.' At the bottom of the form are two buttons: 'Cancel' and 'Generate'.

Preparation at Github: set secret for github access

leeyoungkon / book-api

Code Issues Pull requests Actions Projects Wiki Security Insights **Settings**

General

Access

- Collaborators
- Moderation options

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models [Preview](#)
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security

- Advanced Security
- Deploy keys
- * Secrets and variables**

General

Repository name

book-api [Rename](#)

☐ **Template repository**

Template repositories let users generate new repositories with the same directory structure and files. [Learn more about template repositories.](#)

Default branch

The default branch is considered the “base” branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

main [Edit](#)

Releases

☐ **Enable release immutability**

Disallow assets and tags from being modified once a release is published.

Social preview

Upload an image to customize your repository’s social media preview.

Images should be at least 640×320px (1280×640px for best display).

[Download template](#)

Preparation at Github: set secret for github access

The screenshot shows the GitHub repository settings page for the repository 'leeyoungkon / book-api'. The 'Settings' tab is selected in the top navigation bar. The left sidebar shows the 'Secrets and variables' section, which is highlighted with a red circle. The 'Secrets' tab is selected, and the 'Repository secrets' table is visible. A 'New repository secret' button is highlighted with a red circle.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets

Name ↕	Last updated
DOCKERHUB_TOKEN	15 minutes ago
DOCKERHUB_USERNAME	19 minutes ago

[New repository secret](#)

Preparation at Github: set secret for github access

leeyoungkon / book-api

Issues Pull requests Actions Projects Wiki Security Insights Settings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

Actions

Actions secrets / New secret

Name *

DOCKERHUB_USERNAME

Secret *

Add secret

Preparation at Github: set secret for github access

The screenshot shows the GitHub interface for the repository 'leeyoungkon / book-api'. The 'Settings' tab is selected, and the 'Actions secrets / New secret' page is displayed. The left sidebar contains a list of settings categories: General, Access, Collaborators, Moderation options, Code and automation, Branches, Tags, Rules, Actions, Models, Webhooks, Copilot, Environments, Codespaces, Pages, Security, Advanced Security, Deploy keys, and Secrets and variables. The 'Secrets and variables' section is expanded, showing 'Actions' as a sub-section. The main content area has a title 'Actions secrets / New secret'. Below the title, there are two input fields: 'Name *' and 'Secret *'. The 'Name *' field contains the text 'DOCKERHUB_TOKEN' and is circled in red. The 'Secret *' field is empty. Below the 'Secret *' field, there is a green button labeled 'Add secret'. A 'Preview' button is also visible next to the 'Models' section in the sidebar.

leeyoungkon / book-api

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

General

Actions secrets / New secret

Name *

DOCKERHUB_TOKEN

Secret *

Preview

Add secret

Secrets and variables

Actions

CI test for Book-api project by github hosted runner

The screenshot shows the GitHub Actions interface for the 'book-api' repository. The 'Actions' tab is selected, displaying a workflow named 'Merge branch 'main' of https://github.com/leeyoungkon/book-api #39'. The workflow status is 'Success', triggered by a push to the 'main' branch 6 minutes ago. The total duration is 1m 29s. The workflow file is 'ci.yml', and the job 'build' is shown as successful with a duration of 1m 25s.

leeyoungkon / book-api

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

← CI

✓ Merge branch 'main' of https://github.com/leeyoungkon/book-api #39

Summary

All jobs

✓ build

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

Status

leeyoungkon pushed 063dc3a main

Success

Total duration

1m 29s

Artifacts

—

ci.yml

on: push

✓ build 1m 25s

Preparation at Github: Determining where to run workflow

- ✓ GitHub Hosted Runner
 - Run on servers provided by GitHub
 - runs-on: ubuntu-latest
 - Flow: GitHub Push → Runner on the GitHub server → Build/Test/Push
 - No local PC required
- ✓ Self-hosted Runner
 - Run on my PC or on-premises server
 - runs-on: self-hosted
 - Flow: GitHub Push → GitHub Forward Tasks to My Runner → Run on My PC
 - Local Runner Required

Ready at Github: If You Need a Self-Hosted Runner

- ✓ If you have resources that the GitHub server does not have direct access to:
 - In-house Harbour
 - Internal Kubernetes
 - Private network DB
 - Docker Desktop
 - On-premises infrastructure

Preparation at Github: self-hosted runner register ⇒ execute cmd

Runners / Add new self-hosted runner · leeyoungkon/resttemplateserver-hub

Add new self-hosted runner · leeyoungkon/resttemplateserver-hub

Warning: Using self-hosted runners in public repositories is not recommended. Forks of your public repository can potentially run dangerous code on your self-hosted runner by creating a pull request. [Learn more about security hardening for self-hosted runners.](#)

Adding a self-hosted runner requires that you download, configure, and execute the GitHub Actions Runner. If you do not already have an existing volume licensing agreement for your GitHub purchases, by downloading and configuring the GitHub Actions Runner, you agree to the [GitHub Customer Agreement](#).

Runner image

☐ macOS ☐ Linux ☒ Windows

Architecture

Download

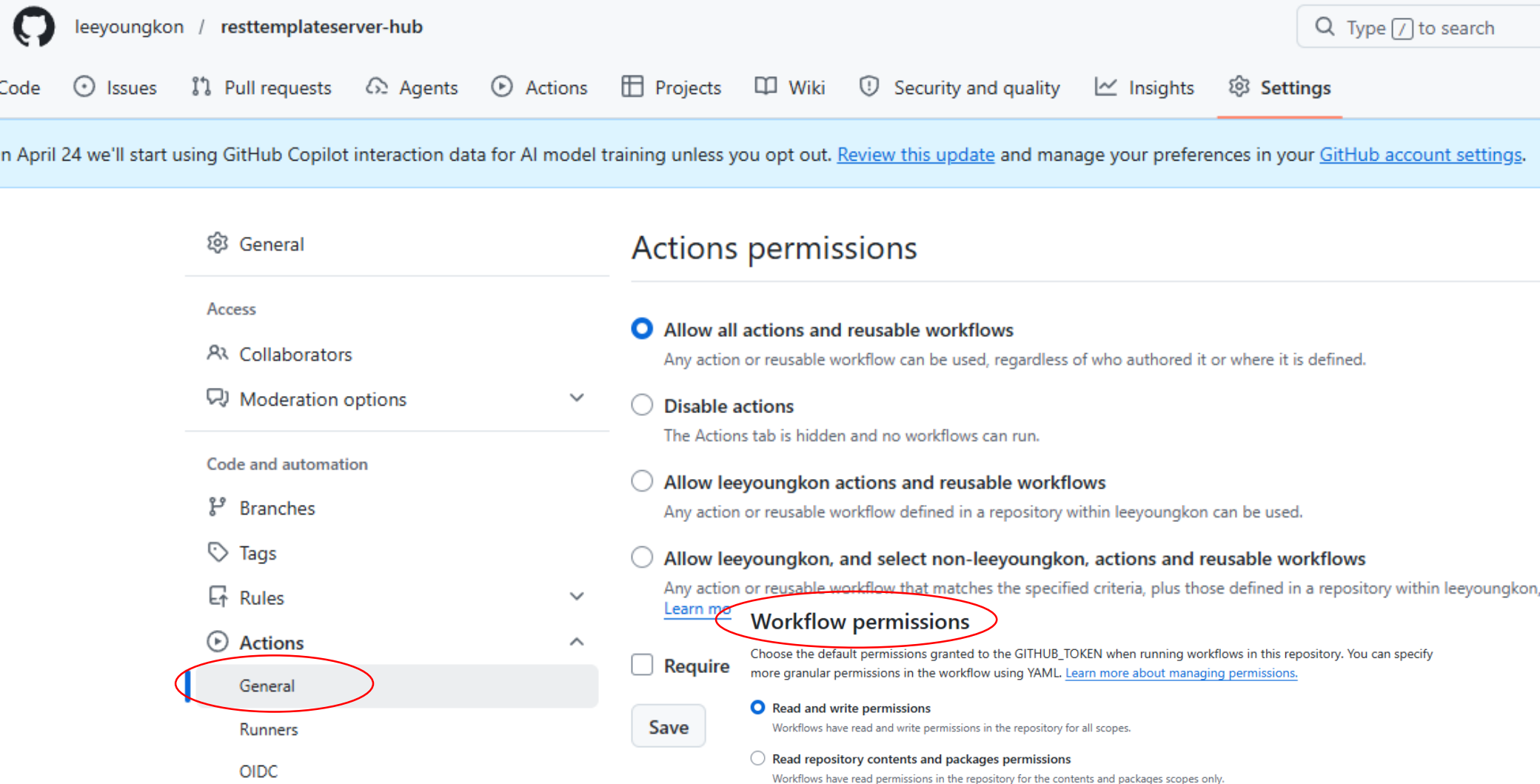
We recommend configuring the runner under "actions-runner". This will help avoid issues related to service identity folder permissions and long path restrictions on Windows.

```
# Create a folder under the drive root
$ mkdir actions-runner; cd actions-runner

# Download the latest runner package
$ Invoke-WebRequest -Uri https://github.com/actions/runner/releases/download/v2.334.0/actions-runner-win-x64-2.334.0.zip -OutFile actions-runner-win-x64-2.334.0.zip

# Optional: Validate the hash
$ if((Get-FileHash -Path actions-runner-win-x64-2.334.0.zip -Algorithm SHA256).Hash.ToUpper() -ne 'a0c896f3acf37841cc17f392a38111d39501e56f2990434567f027ee89cf8981'.ToUpper()){ throw 'Computed checksum did not match expected checksum'
```

Preparation at Github: action $\Rightarrow$ general $\Rightarrow$ W/F permission grant



leeyoungkon / resttemplateserver-hub

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

General

Runners

OIDC

Actions permissions

☒ **Allow all actions and reusable workflows**
Any action or reusable workflow can be used, regardless of who authored it or where it is defined.

☐ **Disable actions**
The Actions tab is hidden and no workflows can run.

☐ **Allow leeyoungkon actions and reusable workflows**
Any action or reusable workflow defined in a repository within leeyoungkon can be used.

☐ **Allow leeyoungkon, and select non-leeyoungkon, actions and reusable workflows**
Any action or reusable workflow that matches the specified criteria, plus those defined in a repository within leeyoungkon. [Learn more](#)

Workflow permissions

☐ **Require**
Choose the default permissions granted to the GITHUB_TOKEN when running workflows in this repository. You can specify more granular permissions in the workflow using YAML. [Learn more about managing permissions.](#)

☒ **Read and write permissions**
Workflows have read and write permissions in the repository for all scopes.

☐ **Read repository contents and packages permissions**
Workflows have read permissions in the repository for the contents and packages scopes only.

Save

Execute CI workflow by actions-runner

The screenshot displays the GitHub Actions interface for the repository 'leeyoungkon / book-api'. The 'Actions' tab is selected and highlighted with a red circle. Below the repository name, the workflow 'Enhance CI workflow for Docker image handling #17' is shown with a green checkmark. The left sidebar contains a 'Summary' section with 'All jobs' and a list of jobs: 'build' (marked with a green checkmark), 'Run details', 'Usage', and 'Workflow file'. The main content area shows the workflow details for 'ci.yml' on the 'push' event. A table lists the jobs, with 'build' circled in red, showing a status of 'Success' and a duration of '1m 10s'. The 'Annotations' section at the bottom indicates '1 warning'.

leeyoungkon / book-api

<> Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

← CI

✓ Enhance CI workflow for Docker image handling #17

Summary

All jobs

✓ build

Run details

Usage

Workflow file

Re-run triggered 7 minutes ago

Status

Total duration

Artifacts

leeyoungkon - fe9069 main Success 1m 14s -

ci.yml

on: push

✓ build 1m 10s

Annotations

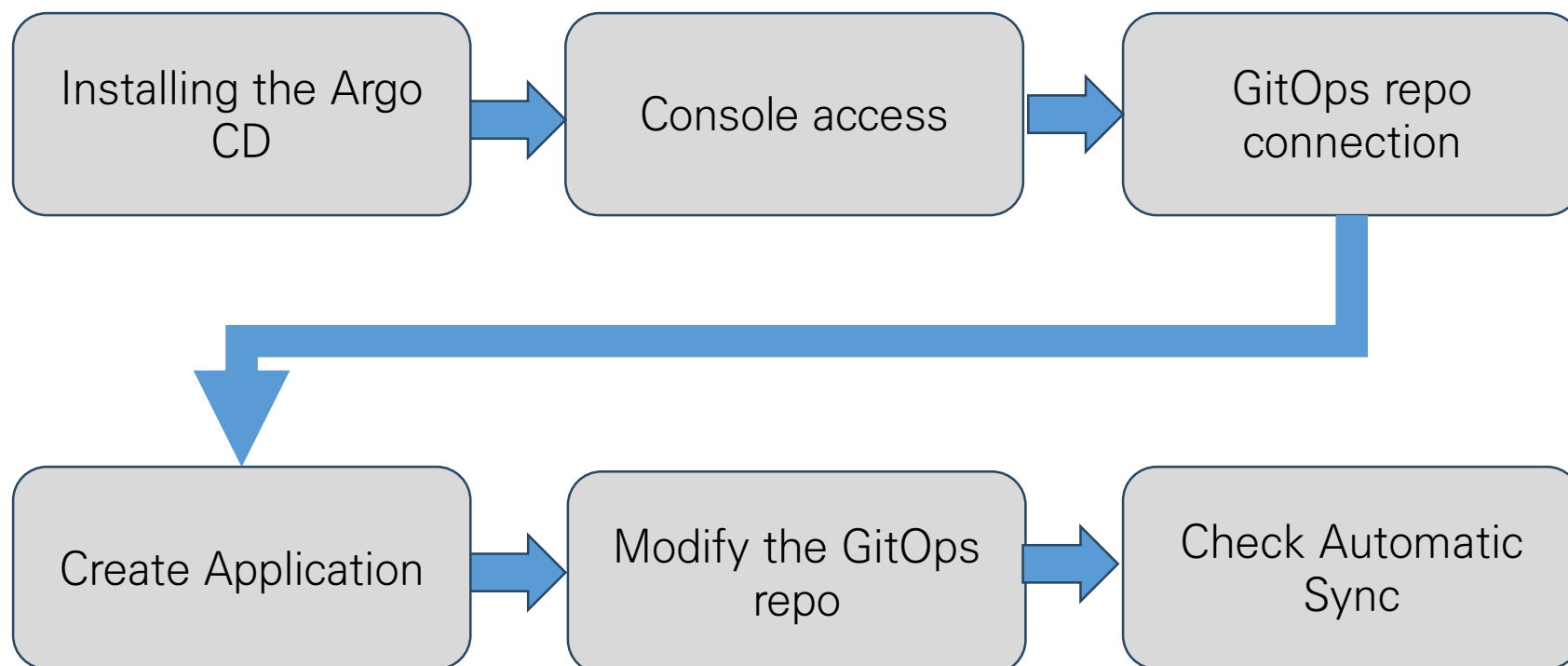
1 warning

Confirm docker image at docker hub

The screenshot shows the Docker Hub interface for the user 'yklee2002'. The 'Repositories' tab is selected in the left sidebar. The main content area displays a list of repositories within the 'yklee2002' namespace. The repository 'yklee2002/book-api' is circled in red. The table below summarizes the visible repositories:

Name	Last Pushed	Contains	Visibility	Scout
yklee2002/book-api	10 minutes ago	IMAGE	Public	Inactive
yklee2002/demo	4 days ago	IMAGE	Public	Inactive
yklee2002/msa	5 days ago	IMAGE	Public	Inactive
yklee2002/second	20 days ago	IMAGE	Public	Inactive
yklee2002/receiver	3 months ago	IMAGE	Public	Inactive
yklee2002/sender	3 months ago	IMAGE	Public	Inactive
yklee2002/user	4 months ago	IMAGE	Public	Inactive
yklee2002/maker	4 months ago	IMAGE	Public	Inactive

Process



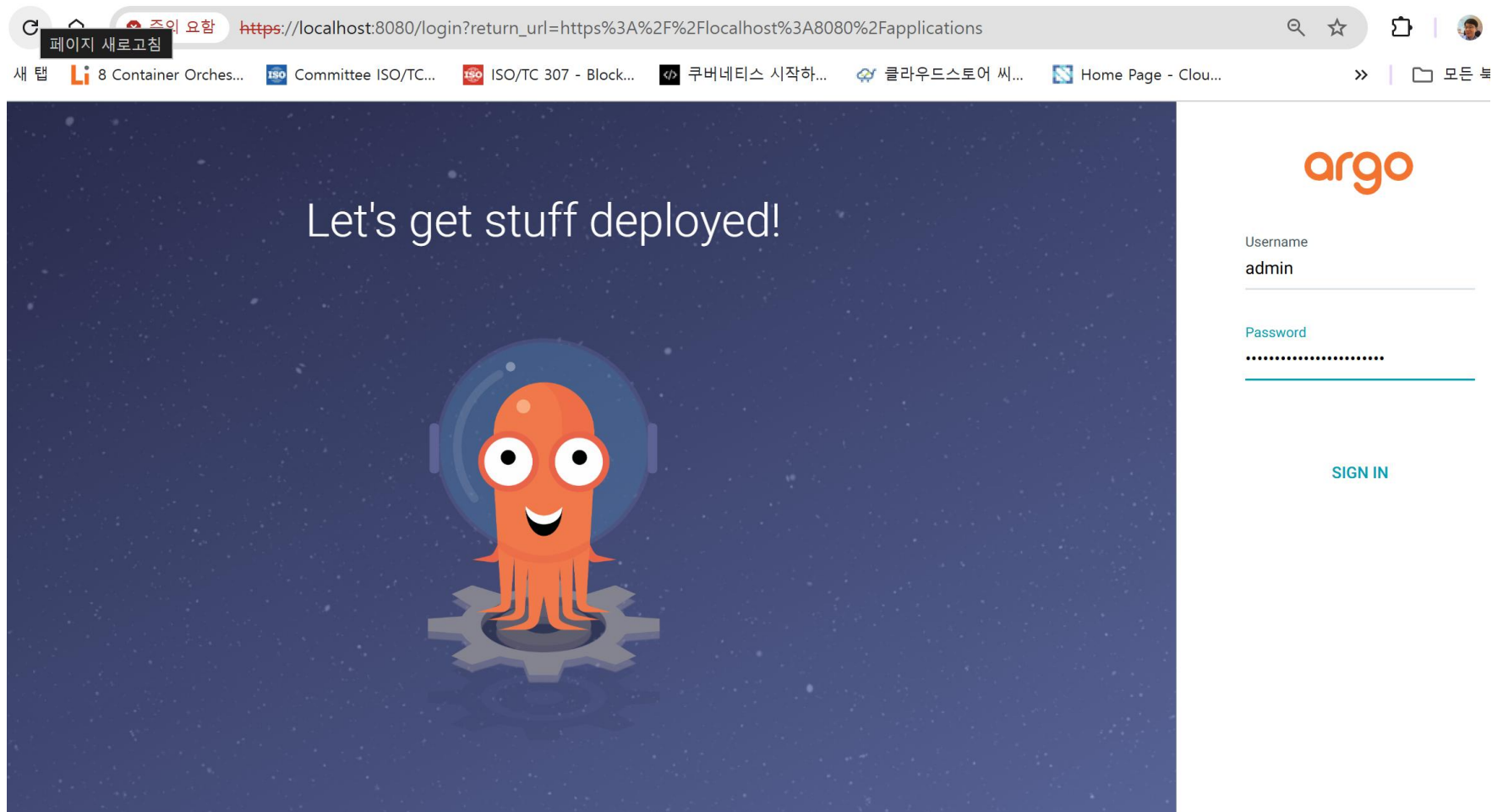
Check the context for ArgoCD installation

- ✓ `kubectl config current-context //`Check current context
- ✓ `check kubectl get nodes //`node
- ✓ `kubectl config get-contexts //` View context information
- ✓ `kubectl cluster-info //`View cluster information currently in use
- ✓ `kubectl config view //`Config view view
- ✓ `convert kubectl config use-context default //`context to default

Installing the Argo CD

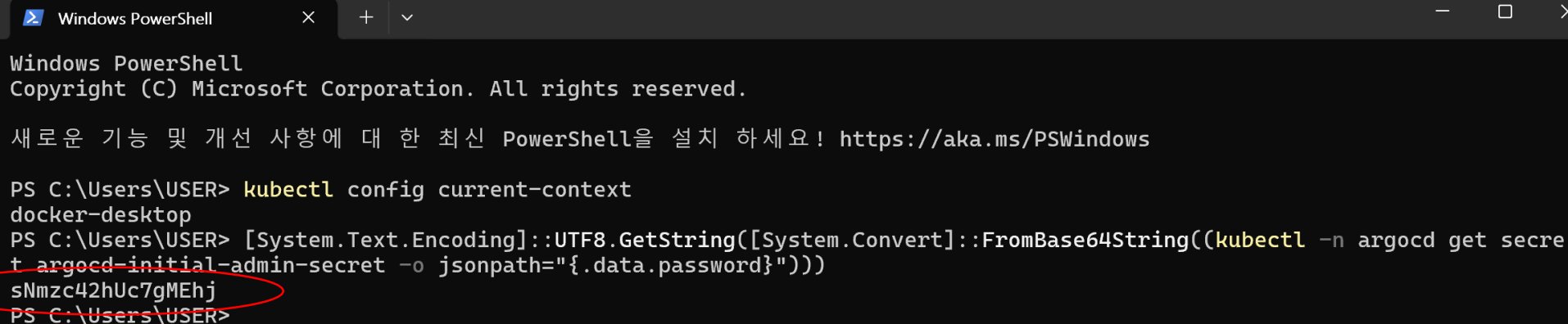
- ✓ `kubectl create namespace argocd`
- ✓ `kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml`
- ✓ `kubectl get pods -n argocd`
- ✓ `kubectl port-forward svc/argocd-server -n argocd 8080:443`
- ✓ `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d && echo //linux server`
- ✓ `[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}"))) //windows powershell`

ArgoCD UI 접속: localhost:8080



Log in to ArgoCD

- ✓ ID: admin
- ✓ Password: Enter the password generated by powershell as shown below



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

새로운 기능 및 개선 사항에 대한 최신 PowerShell을 설치 하세요! https://aka.ms/PSWindows

PS C:\Users\USER> kubectl config current-context
docker-desktop
PS C:\Users\USER> [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl -n argocd get secret
t argocd-initial-admin-secret -o jsonpath="{.data.password}")))
sNmzc42hUc7gMEhj
PS C:\Users\USER>
```

CD application process to developed apps

The screenshot shows the ArgoCD web interface. The browser address bar indicates the URL is `https://localhost:8080/applications`. The left sidebar features the Argo logo (v3.3.3) and navigation links: Applications, Settings, User Info, and Documentation. The main content area is titled 'Applications' and includes buttons for '+ NEW APP', 'SYNC APPS', and 'REFRESH APPS'. A search bar is also present. The main area displays a large graphic of stacked diamonds and the message: 'No applications available to you just yet. Create new application to start managing resources in your cluster.' A 'CREATE APPLICATION' button is at the bottom right.

Create an Application from ArgoCD

- ✓ Name: book-api-dev
- ✓ Project: default
- ✓ Sync policy: manual (first)
- ✓ Repository URL: <https://github.com/yourid/book-api.git>
- ✓ Revision: HEAD
- ✓ Path: .
- ✓ Cluster URL: <https://kubernetes.default.svc>
- ✓ Namespace: default (or namespace you made)
- ✓ Press the create button at the end

Application sync on ArgoCD

The screenshot displays the ArgoCD web interface in a browser. The address bar shows the URL: `https://localhost:8080/applications?showFavorites=false&proj=&sync=&autoSync=&health=&namespace=&cluster=&labels=`. The browser's tab bar includes several open tabs, such as "주요 요약", "8 Container Orches...", "Committee ISO/TC...", "ISO/TC 307 - Block...", "쿠버네티스 시작하...", "클라우드스토어 씨...", "Home Page - Clou...", and "[시장동향] 대세는...".

The ArgoCD interface features a dark sidebar on the left with the following menu items: "argo v3.3.3", "Applications", "Settings", "User Info", and "Documentation". Below these are "Application filters" and a "Favorites Only" toggle. The "SYNC STATUS" section shows a list of application sync states: "Unknown" (0), "Synced" (1), and "OutOfSync" (0). The "HEALTH STATUS" section shows "Progressing" (0).

The main content area, titled "Applications", contains a toolbar with buttons for "+ NEW APP", "SYNC APPS", "REFRESH APPS", and a search bar. A card for the application "book-api-dev" is displayed. The card's header includes the application icon and name. A red circle highlights the "Sync" icon (a circular arrow) in the top right corner of the card. The card's body lists the following details:

- Project: default
- Labels:
- Status: ♥ Healthy ✔ Synced
- Repository: `https://github.com/leeyoungkon/book-api.git`
- Target Revi...: HEAD
- Path: `overlays/dev`
- Destination: in-cluster
- Namespace: book-dev
- Created At: 03/13/2026 12:36:39 (26 minutes ago)
- Last Sync: 03/13/2026 13:00:08 (2 minutes ago)

At the bottom of the card, there are three buttons: "SYNC", "REFRESH", and "DELETE".

Verifying the Running Status on ArgoCD

← → ↺ 🏠 주의 요함 <https://localhost:8080/applications/argocd/book-api-dev?view=tree&resource=>

📦 새 탭 8 Container Orches... ISO/TC 307 - Block... 쿠버네티스 시작하... 클라우드스토어 써... Home Page - Clou... [시장동향] 대세는... 큰 숲 맑은 샘 대한성서



- Applications
- Settings
- User Info
- Documentation

Resource filters

NAME

KINDS

SYNC STATUS

HEALTH STATUS

Applications / Q book-api-dev

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH  **Healthy**

SYNC STATUS  **Synced** to HEAD (af9cb53) [🔗](#)

Auto sync is not enabled.
Author: leeyoungkon <yklee2002@gmail.com> -
Comment: libe probe 지움

LAST SYNC  **Sync OK** to af9cb53 [🔗](#)

Succeeded 4 minutes ago (Fri Mar 13 2026 13:00:08 GMT+0900)
Author: leeyoungkon <yklee2002@gmail.com> -
Comment: libe probe 지움

100%

```

graph LR
    book-api-dev[book-api-dev] --> book-dev[book-dev]
    book-api-dev --> book-api-svc[book-api]
    book-api-dev --> book-api-deploy[book-api]
    book-api-deploy --> book-api-576cf7b76f[book-api-576cf7b76f]
    book-api-deploy --> book-api-6854668db[book-api-6854668db]
  
```

book-api-dev (27 minutes)

book-dev (27 minutes)

book-api (27 minutes)

book-api (27 minutes) rev:2

book-api-576cf7b76f (4 minutes) rev:2

book-api-6854668db (27 minutes) rev:1

Also see in the command window

```
C:\Users\leeyoungkon>kubectl get all -n book-dev
```

NAME	READY	STATUS	RESTARTS	AGE
pod/book-api-576cf7b76f-kk5m7	1/1	Running	0	10s
pod/book-api-576cf7b76f-qzq8p	1/1	Running	0	21s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/book-api	ClusterIP	10.110.16.218	<none>	80/TCP	23m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/book-api	2/2	2	2	23m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/book-api-576cf7b76f	2	2	2	21s
replicaset.apps/book-api-6854668db	0	0	0	23m

Auto sync settings on ArgoCD: click details

← → ↺ 🏠 주의 요함 <https://localhost:8080/applications/argocd/book-api-dev?view=tree&resource=>

📦 새 탭 8 Container Orches... 100 Committee ISO/TC... 100 ISO/TC 307 - Block... <📁> 쿠버네티스 시작하... 🌐 클라우드스토어 써... 🏠 Home Page - Clou... 📊 [시장동향] 대세는... 📈 큰 숲 맑은 샘 🌐 대한성서

argo v3.3.3

Applications Settings User Info Documentation

Resource filters

NAME

NAME

KINDS

KINDS

SYNC STATUS

☐ Synced 3

☐ OutOfSync 0

HEALTH STATUS

☐ Progressing 0

Applications / Q book-api-dev

DETAILS DIFF SYNC SYNC STATUS HISTORY AND ROLLBACK DELETE REFRESH

APP HEALTH **Healthy**

SYNC STATUS **Synced** to HEAD (af9cb53) [🔗](#)

Auto sync is not enabled.

Author: leeyoungkon <ykle2002@gmail.com> -
Comment: libe probe 지움

LAST SYNC **Sync OK** to af9cb53 [🔗](#)

Succeeded 4 minutes ago (Fri Mar 13 2026 13:00:08 GMT+0900)

Author: leeyoungkon <ykle2002@gmail.com> -
Comment: libe probe 지움

☰ ☰ ☰ + - 🔍 100%

book-api-dev 27 minutes

book-dev ns 27 minutes

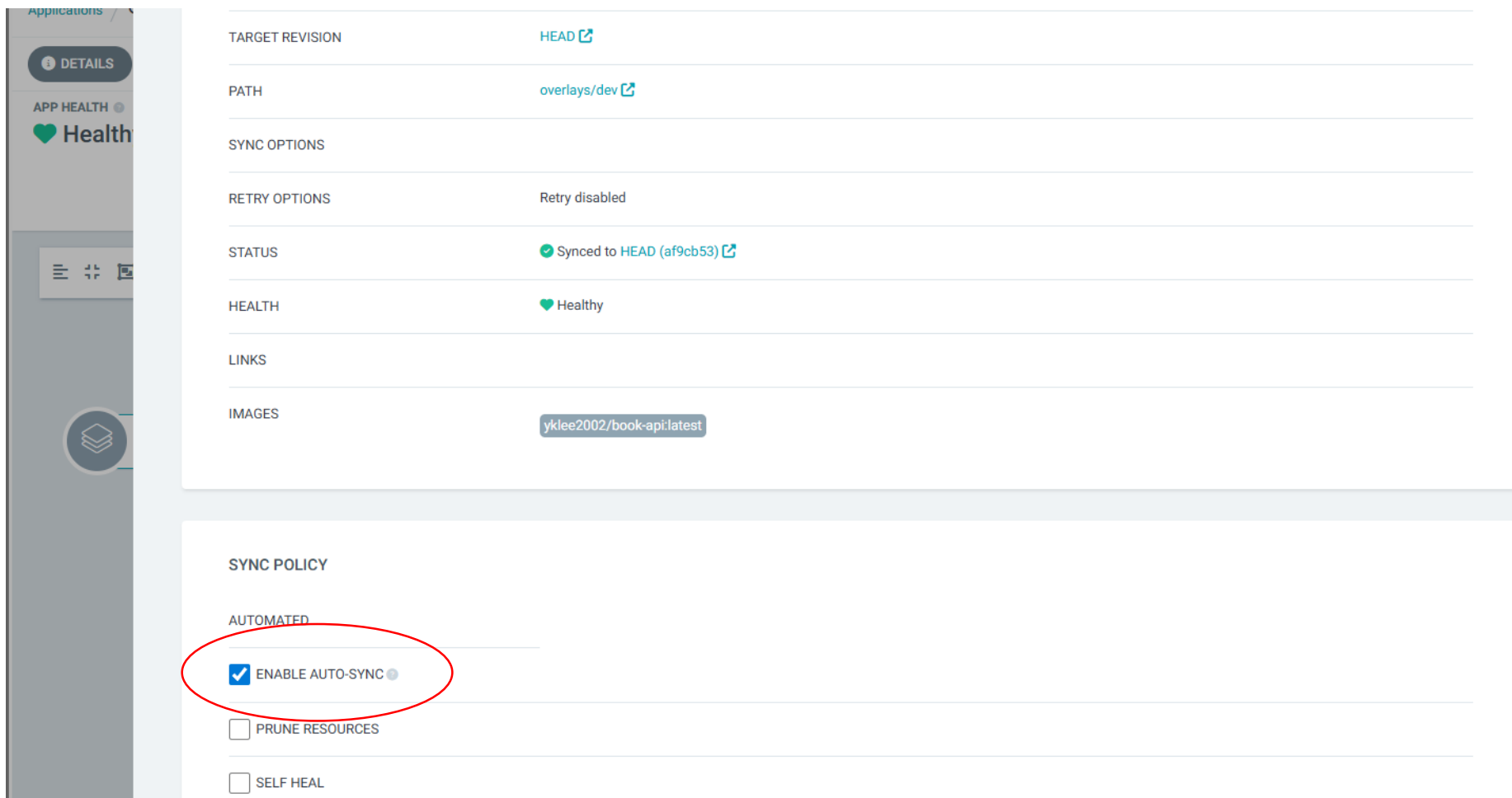
book-api svc 27 minutes

book-api deploy 27 minutes rev:2

book-api-576cf7b76f rs 4 minutes rev:2

book-api-6854668db rs 27 minutes rev:1

Auto sync setting in ArgoCD: Auto sync check in summary tab



The screenshot displays the ArgoCD application summary page. On the left sidebar, the 'DETAILS' tab is selected, and the application health is shown as 'Healthy'. The main content area is divided into two sections. The top section, titled 'SUMMARY', contains the following information:

- TARGET REVISION: HEAD
- PATH: overlays/dev
- SYNC OPTIONS
- RETRY OPTIONS: Retry disabled
- STATUS: Synced to HEAD (af9cb53)
- HEALTH: Healthy
- LINKS
- IMAGES: yklee2002/book-api:latest

The bottom section, titled 'SYNC POLICY', contains the following settings:

- AUTOMATED
 - ☒ ENABLE AUTO-SYNC
 - ☐ PRUNE RESOURCES
 - ☐ SELF HEAL

The 'ENABLE AUTO-SYNC' checkbox is highlighted with a red circle.

Docker Image Registry

✓ Login url: <http://210.90.24.172:8080>, id : admin, pwd : Harbor12345

The screenshot displays the Harbor web interface in the background and a terminal window in the foreground. The terminal shows the command `sudo docker compose up -d` being executed, which successfully starts the Harbor services. The web interface shows the '프로젝트' (Projects) page with a table listing the 'library' project.

Terminal Output:

```
ubuntu@ubuntu:~/harbor$ sudo docker compose up -d
[+] Running 10/10
✓ Network harbor_harbor      Created
✓ Container harbor-log       Started
✓ Container harbor-db        Started
✓ Container registry         Started
✓ Container registryctl      Started
✓ Container redis            Started
✓ Container harbor-portal    Started
✓ Container harbor-core      Started
✓ Container harbor-jobservice Started
✓ Container nginx            Started
```

Harbor Web Interface:

- Search: 210.90.24.172:8080/harbor/projects
- Left Menu: 프로젝트, 로그, 관리, 사용자, 로봇 계정, 레지스트리, 복제, 배포, 라벨, 프로젝트 할당량, 질의 서비스, 정리, 작업 서비스 대시보드, 설정
- Main Content: 프로젝트
- Buttons: + 새 프로젝트, 동작
- Table:

☐	프로젝트 이름	엑세스 레벨	역할	종류	저장소 수	생성 시간
☐	library	공개	프로젝트 관리자	프로젝트	0	26. 5. 1. PM 11:25

페이지 크기: 15, 1 - 1 of 1 아이템

Docker Image Registry

- ✓ After you create a project, you must register a user and register that user with the project
- ✓ Project Repository is automatically generated when you first push an image

Harbor

검색 Harbor...

사용자

+ 새 사용자 관리자로 설정 동작 ▼

☐	이름	관리자 ⓘ
☐	leeyoungkon	아니요
☐	yklee2002	아니요

CI.yaml (change with Harbour use)

```
env:  
  HARBOR_REGISTRY: 210.90.24.172:8080  
  HARBOR_PROJECT: restproject  
  STUDENT_ID: leeyoungkon  
  IMAGE_NAME: resttemplateserver
```



repository name

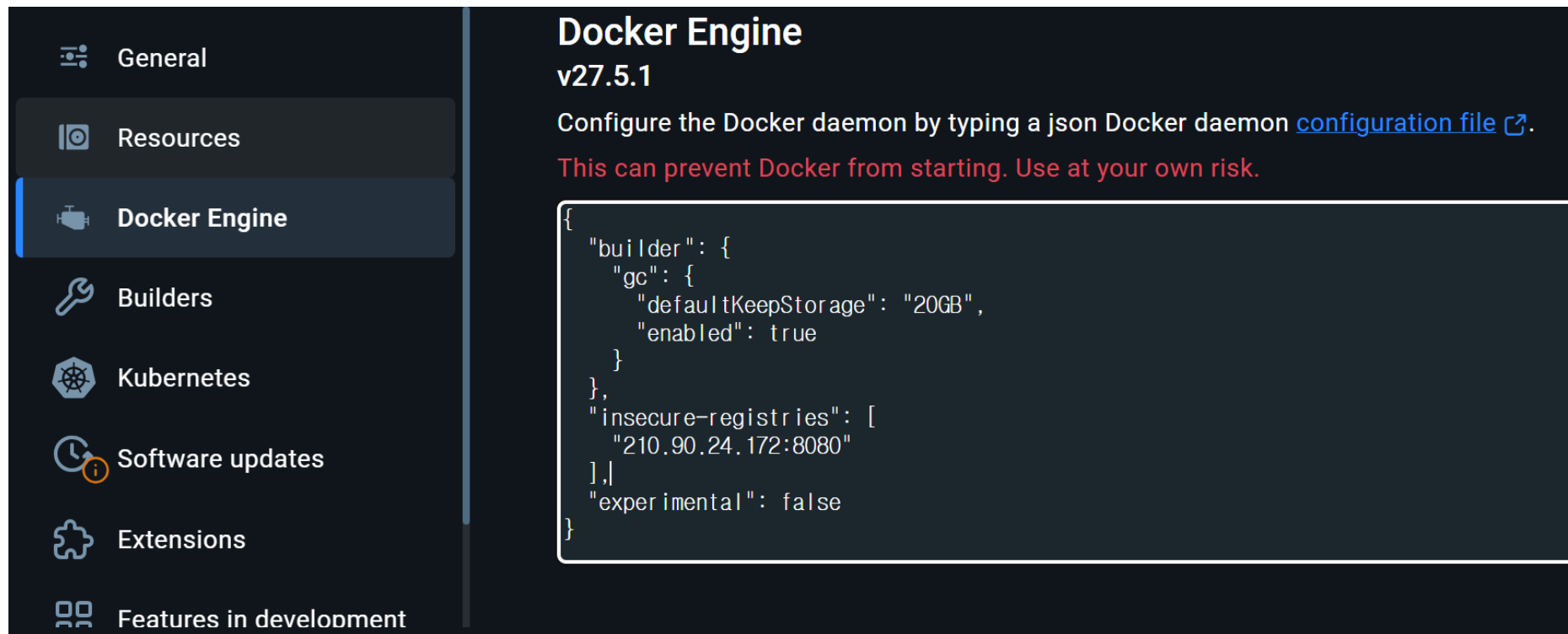
```
registry: ${{ env.HARBOR_REGISTRY }}  
username: ${{ secrets.HARBOR_USERNAME }}  
password: ${{ secrets.HARBOR_PASSWORD }}  
  
name: Build and push Docker image  
uses: docker/build-push-action@v5  
with:  
  context: .  
  push: true  
  tags: |  
    ${{ env.HARBOR_REGISTRY }}/${{ env.HARBOR_PROJECT }}/${{ env.STUDENT_ID }}/${{ env.IMAGE_NAME }}  
    ${{ env.HARBOR_REGISTRY }}/${{ env.HARBOR_PROJECT }}/${{ env.STUDENT_ID }}/${{ env.IMAGE_NAME }}
```

k8s.yaml change part

```
spec:
  containers:
  - name: resttemplateserver
    image: 210.90.24.172:8080/restproject/leeyoungkon/resttemplateserver:eddda1e8bf5d9538f
    imagePullPolicy: Always
    ports:
    - containerPort: 8080
```

[Harbor registry ip] [Project Name] [Repository Name] [Image Name: SHA Tag]

Add Insecure Registry Information to Docker Engine



Docker Engine
v27.5.1

Configure the Docker daemon by typing a json Docker daemon [configuration file](#).
This can prevent Docker from starting. Use at your own risk.

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
      "enabled": true
    }
  },
  "insecure-registries": [
    "210.90.24.172:8080"
  ],
  "experimental": false
}
```

To register Harbour as an insecure registry and access it to http

```
"insecure-registries": [
  "210.90.24.172:8080"
]
```


In k8s.yaml, harbor secret is required to get docker image in Harbor

```

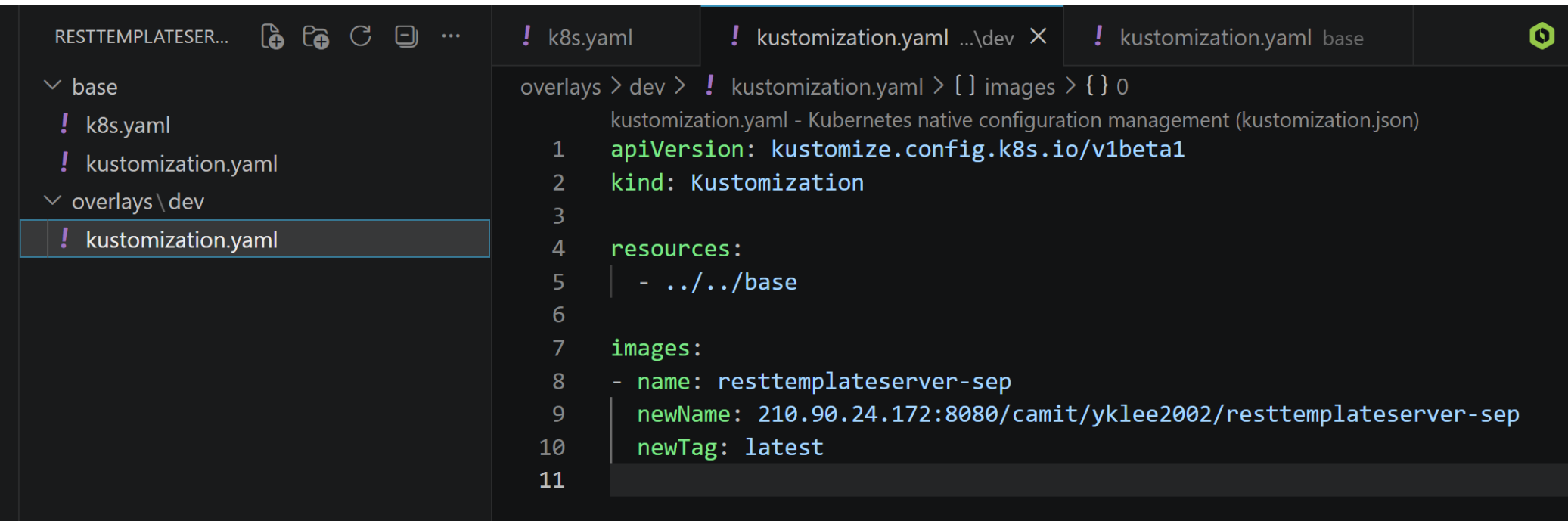
✓ base
! k8s.yaml
! kustomization.yaml
✓ overlays\dev
! kustomization.yaml

base > ! k8s.yaml > {} spec > {} selector > {} matchLabels > abc app
3  metadata:
4    name: resttemplateserver-sep
5    labels:
6      app: resttemplateserver-sep
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: resttemplateserver-sep
12  template:
13    metadata:
14     labels:
15       app: resttemplateserver-sep
16    spec:
17     imagePullSecrets:
18       - name: harbor-secret
19
20     containers:
21       - name: resttemplateserver-sep
22         image: 210.90.24.172:8080/camit/yklee2002/resttemplateserver-sep:1
23         imagePullPolicy: Always

```

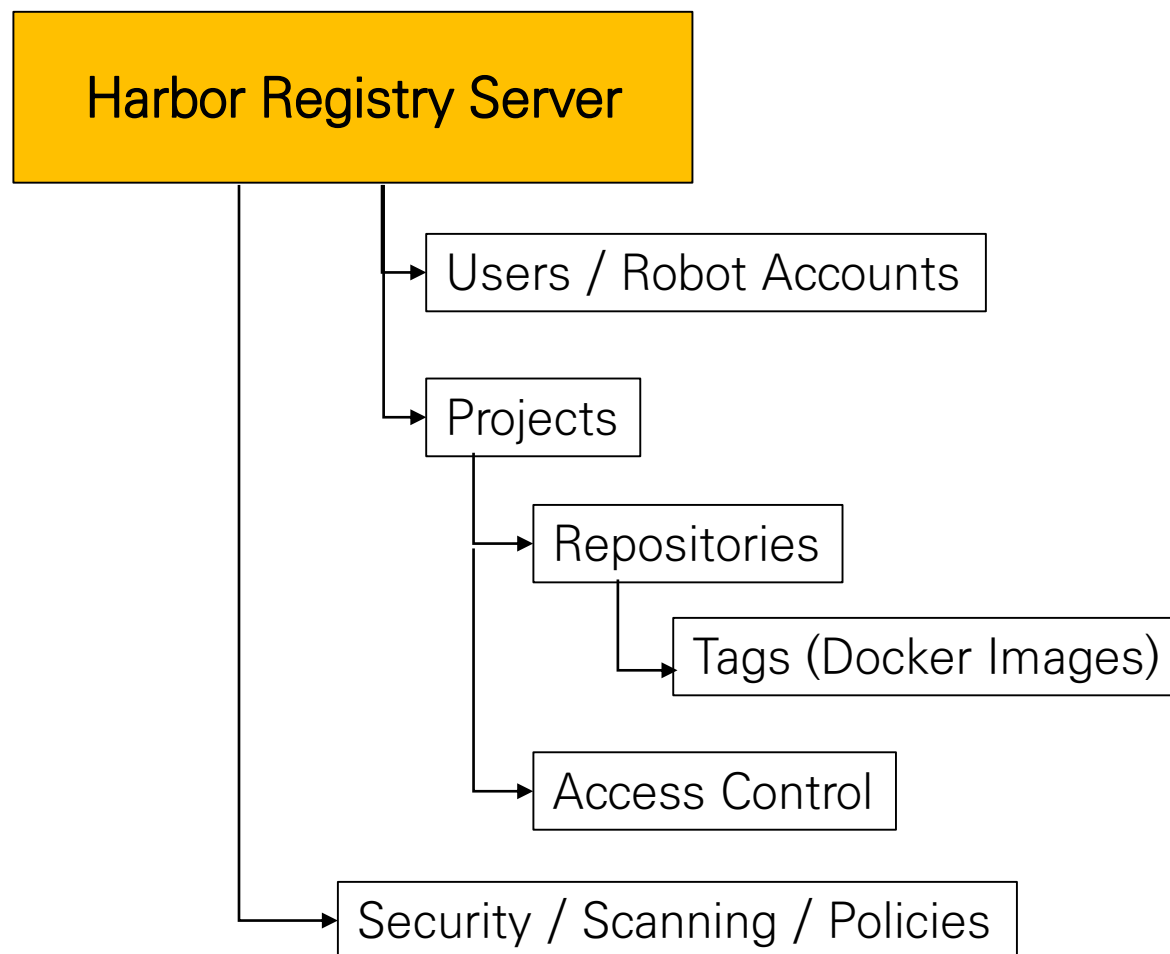
```
kubectl create secret docker-registry harbor-secret --docker-server=210.90.24.172:8080 --
docker-username=yklee2002 --docker-password=Byby3845// --docker-
email=yklee2002@gmail.com -n default
```

In kustomization.yaml, resources and images should be included.



```
RESTTEMPLATESE...  ! k8s.yaml  ! kustomization.yaml ...\dev X  ! kustomization.yaml base  !  
▼ base  
  ! k8s.yaml  
  ! kustomization.yaml  
▼ overlays\dev  
  ! kustomization.yaml  
overlays > dev > ! kustomization.yaml > [ ] images > { } 0  
kustomization.yaml - Kubernetes native configuration management (kustomization.json)  
1  apiVersion: kustomize.config.k8s.io/v1beta1  
2  kind: Kustomization  
3  
4  resources:  
5  | - ../../base  
6  
7  images:  
8  - name: resttemplateserver-sep  
9    newName: 210.90.24.172:8080/camit/yklee2002/resttemplateserver-sep  
10   newTag: latest  
11
```

Harbor Configuration



Harbor Element Description

Element	Necessity	Description
Harbor server (Registry)	Essential	Full Storage
Project	Essential	push target
User/Robot Account	Essential	Certified.
Project Permissions	Essential	Allow push
GitHub Secrets	Essential	CD Authentication
Repository	Automatic generation	The first push

Applications /

APPLICATION DETAILS

DETAILS

DIFF

SYNC

SYNC STATUS

HISTORY AND ROLLBACK

DELETE

REFRESH



APP HEALTH

Healthy

SYNC STATUS



OutOfSync from HEAD (931ea2f)

Auto sync is not enabled.

Author: rkgj4106 <rkgj0426@gmail.com> -
Comment: 8th commit

LAST SYNC



Sync OK to 931ea2f

Succeeded 8 minutes ago (Mon Aug 03 2026
02:26:14 GMT+0900)Author: rkgj4106 <rkgj0426@gmail.com> -
Comment: 8th commit

er-sep

38 minutes

rev:2

resttemplateserver-sep-
86674cb9cc

21 minutes

rev:2

resttemplateserver-sep-
7d8fdcddb

38 minutes

rev:1

resttemplateserver-sep-
86674cb9cc-qkpfw

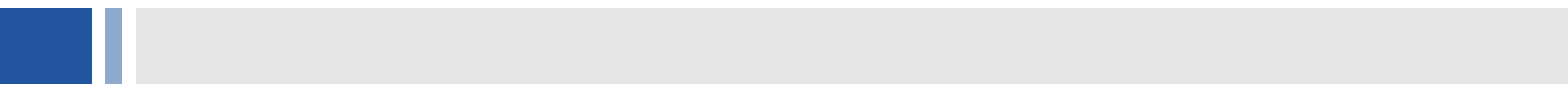
pod

21 minutes

running

1/1

more



Thank You